

5CS022 Distribute and Cloud Systems Programming Week 2 Workshop

Overview

The aim of this workshop is to build on the previous workshop with MPI and to carry out an assessed task. You can carry out this workshop either the university servers: thinlinc.wlv.ac.uk or your own Linux system (if you prefer to use Putty to log in instead of the Thinlinc client, connect to the **server tl-01.wlv.ac.uk** instead). **Tasks**

1. The following C program sums up all the values in array "data" and displays the sum total.:

```
#include <stdio.h>

#define NUMDATA 10000 int data[NUMDATA];

void LoadData(int data[])
{ for(int i = 0; i < NUMDATA; i++){
    data[i] = 1;
  }
}

int AddUp(int data[], int count)
{ int sum = 0;
  for(int i = 0; i < count; i++){
    sum += data[i];
  }
  return sum;
}

int main(void) { int sum;

  LoadData(data); sum = AddUp(data,
  NUMDATA);
  printf("The total sum of data is %d\n", sum); return 0;
}
```

Convert it to MPI to run with any number of nodes including just one.

```

GNU nano 7.2 W1.c
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

#define NUMDATA 10000

void LoadData(int data[], int size) {
    for (int i = 0; i < size; i++) {
        data[i] = 1;
    }
}

int main(int argc, char *argv[]) {
    int rank, size;
    int *data = NULL;
    int local_sum = 0, global_sum = 0;
    int local_size, remainder = 0;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    local_size = NUMDATA / size;
    remainder = NUMDATA % size;

    int *local_data = (int *)malloc(local_size * sizeof(int));

    if (rank == 0) {
        data = (int *)malloc(NUMDATA * sizeof(int));
        LoadData(data, NUMDATA);
    }

    MPI_Scatter(data, local_size, MPI_INT, local_data, local_size, MPI_INT, 0, MPI_COMM_WORLD);

    for (int i = 0; i < local_size; i++) {
        local_sum += local_data[i];
    }

    if (rank == 0) {
        for (int i = NUMDATA - remainder; i < NUMDATA; i++) {
            local_sum += data[i];
        }
    }

    MPI_Reduce(&local_sum, &global_sum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        printf("The total sum of data is %d\n", global_sum);
        free(data);
    }

    free(local_data);
    MPI_Finalize();

    return 0;
}

ojashshakya@LenovoLQ:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/
Distributed and Cloud Systems Programming/Week 2$ mpicc W1.c -o W1
ojashshakya@LenovoLQ:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/
Distributed and Cloud Systems Programming/Week 2$ mpiexec ./W1
The total sum of data is 10000
ojashshakya@LenovoLQ:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/
Distributed and Cloud Systems Programming/Week 2$ mpiexec -n 4 ./W1
The total sum of data is 10000
ojashshakya@LenovoLQ:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/
Distributed and Cloud Systems Programming/Week 2$ mpiexec --oversubscribe -n 10 ./W1
The total sum of data is 10000
ojashshakya@LenovoLQ:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/
Distributed and Cloud Systems Programming/Week 2$ nano W1.c

```

Explanation: This code distributes the task of adding up 10,000 elements in an array among several processes using MPI parallel processing. It gives advantage of size and rank. The MPI_Scatter Function is used to partition the whole data into discrete parts. Every procedure uses the AddUp function to determine the total amount of the received chunk. MPI_Reduce is subsequently used to combine these parts.

2. Write an MPI program called pingpong.c to run with exactly 2 processes. Process rank 0 is to send an integer variable called "ball" initialised with the value zero to Process rank 1. Process rank 1 will add 1 to the ball and send it back. This will repeat until the ball has a value of 10 in Process rank 0.

```
GNU nano 7.2 pingpong.c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char *argv[]) {
    int rank, ball = 0;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (rank == 0) {
        while (ball < 10) {
            MPI_Send(&ball, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
            printf("Process 0 sent ball = %d to Process 1\n", ball);
            MPI_Recv(&ball, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            printf("Process 0 received ball = %d from Process 1\n", ball);
        }
        printf("Process 0: Ball reached 10, stopping.\n");
    } else if (rank == 1) {
        while (1) {
            MPI_Recv(&ball, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            if (ball >= 10) break;
            ball++;
            printf("Process 1 incremented ball to %d and sent it back\n", ball);
            MPI_Send(&ball, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
        }
    }

    MPI_Finalize();
    return 0;
}
```

```
ojashshakya@LenovoLOQ:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/
Distributed and Cloud Systems Programming/Week 2$ ls
'5CS022 Week 2 MPI Workshop.pdf'  W1  WarAndPeace.txt  pingpong.c
NumbersForMPI.txt                 W1.c  pingpong      '~$S022 Week 2 MPI Workshop.pdf'
ojashshakya@LenovoLOQ:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/
Distributed and Cloud Systems Programming/Week 2$ mpicc pingpong.c -o pingpong
ojashshakya@LenovoLOQ:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/
Distributed and Cloud Systems Programming/Week 2$ mpiexec ./pingpong
Process 0 sent ball = 0 to Process 1
Process 0 received ball = 1 from Process 1
Process 0 sent ball = 1 to Process 1
Process 0 received ball = 2 from Process 1
Process 0 sent ball = 2 to Process 1
Process 0 received ball = 3 from Process 1
Process 0 sent ball = 3 to Process 1
Process 0 received ball = 4 from Process 1
Process 0 sent ball = 4 to Process 1
Process 0 received ball = 5 from Process 1

Process 0: Ball reached 10, stopping.
Process 1 incremented ball to 1 and sent it back
Process 1 incremented ball to 2 and sent it back
Process 1 incremented ball to 3 and sent it back
Process 1 incremented ball to 4 and sent it back
Process 1 incremented ball to 5 and sent it back
Process 1 incremented ball to 6 and sent it back
Process 1 incremented ball to 7 and sent it back
Process 1 incremented ball to 8 and sent it back
Process 1 incremented ball to 9 and sent it back
Process 1 incremented ball to 10 and sent it back
```

Explanation: This MPI program uses message passing to mimic a game of ping-pong between two processes, ranked 0 and 1. Process 0 initiates the process by sending a ball to Process 1, which uses MPI_Send and MPI_Recv to increase the ball and return it back. This conversation keeps going until the ball hits MAX_COUNT 10.

3. Write a "Pass-the-parcel" MPI program that will run with 3 or more nodes, such that Process rank 0 will send an integer variable call "parcel" initialised with 1, to Process rank 1 which will add 1 to the parcel and then send it to Process rank 2, and so on until the highest rank process will send it back to Process rank 0, at which point the parcel variable should contain the value of the number of nodes there are.

```
GNU nano 7.2 Pass-the-parcel.c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char* argv[]) {
    int rank, size, parcel;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size < 3) {
        if (rank == 0) {
            printf("This program requires at least 3 processes to run.\n");
        }
        MPI_Finalize();
        return 1;
    }

    if (rank == 0) {
        parcel = 1; // Initialize parcel
        MPI_Send(&parcel, 1, MPI_INT, rank + 1, 0, MPI_COMM_WORLD);
        MPI_Recv(&parcel, 1, MPI_INT, size - 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Final parcel value at Process %d: %d\n", rank, parcel);
    } else {
        MPI_Recv(&parcel, 1, MPI_INT, rank - 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        parcel += 1;
        int next = (rank + 1) % size;
        MPI_Send(&parcel, 1, MPI_INT, next, 0, MPI_COMM_WORLD);
    }

    MPI_Finalize();
    return 0;
}

ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/
Distributed and Cloud Systems Programming/Week 2$ mpicc Pass-the-parcel.c -o Pass-the-parcel
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/
Distributed and Cloud Systems Programming/Week 2$ mpiexec -n 4 Pass-the-parcel
Final parcel value at Process 0: 4
```

Explanation: With the help of at least three processes, the parcel is passed in this program's implementation of pass-the-parcel, rising in value each time. First, MPI is initialized. After initializing the parcel with 1, process 0 forwards it to process 1, which subsequently obtains the final value from the final process. After receiving the package from the previous rank, each step increases it before moving on to the next one. The final process prints the final value equal to the total number of processes after sending the last packet back to process 0.

4. The following program has all the processes with rank greater than 0 send random amounts of data to Process rank 0 :

```
#include <mpi.h>

#include <stdio.h>
#include <stdlib.h> #define

NUMDATA 1000

int main(int argc, char **argv)
{ int size; int
  rank; int
  tag=0; int
  count;
  MPI_Status status; int
  data[NUMDATA];

  MPI_Init(&argc, &argv);
  MPI_Comm_size(MPI_COMM_WORLD, &size);
  MPI_Comm_rank(MPI_COMM_WORLD, &rank);

  if (rank == 0){ for(int i = 0; i < size - 1; i++) {
    MPI_Recv(data, NUMDATA, MPI_INT, MPI_ANY_SOURCE, MPI_ANY_TAG,
             MPI_COMM_WORLD, &status);
    MPI_Get_count(&status, MPI_INT, &count);
    printf("Node ID: %d; tag: %d; MPI_Get_count: %d; \n", status.MPI_SOURCE,
           status.MPI_TAG, count);
  }
  } else {
    MPI_Send(data, rand()%100, MPI_INT, 0, tag, MPI_COMM_WORLD);
  }
  MPI_Finalize(); return 0;
}
```

Modify it so that it doesn't use a fixed size data buffer but uses "malloc()" to allocate the correct amount of memory to use every time. Note this means that Process rank 0 will need to know what the amount of data it is expecting before it receives it.

```

GNU nano 7.2 W4.c
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

#define NUMDATA 1000

int main(int argc, char **argv)
{
    int size, rank, tag = 0, count;
    MPI_Status status;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (rank == 0) {
        for (int i = 0; i < size - 1; i++) {
            MPI_Probe(MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
            MPI_Get_count(&status, MPI_INT, &count);

            int *data = (int *)malloc(count * sizeof(int));
            MPI_Recv(data, count, MPI_INT, status.MPI_SOURCE, status.MPI_TAG, MPI_COMM_WORLD, &status);

            printf("Node ID: %d; tag: %d; MPI_Get_count: %d; \n", status.MPI_SOURCE, status.MPI_TAG, count);
            free(data);
        }
    } else {
        count = rand() % 100;
        int *data = (int *)malloc(count * sizeof(int));
        MPI_Send(data, count, MPI_INT, 0, tag, MPI_COMM_WORLD);
        free(data);
    }

    MPI_Finalize();
    return 0;
}

ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/
Distributed and Cloud Systems Programming/Week 2$ mpicc W4.c -o W4
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/
Distributed and Cloud Systems Programming/Week 2$ mpiexec -n 4 ./W4
Node ID: 3; tag: 0; MPI_Get_count: 83;
Node ID: 2; tag: 0; MPI_Get_count: 83;
Node ID: 1; tag: 0; MPI_Get_count: 83;

```

Explanation: Process 0 receives data from all other processes as this MPI application dynamically allots memory for data transfer. A random number of data pieces are created by each process (rank > 0), sent to Process 0, and the matching data is subsequently sent. Process 0 gets the data size first, uses malloc() to allocate memory appropriately, and then gets the actual data from each process. Following processing, it releases the allocated memory after printing the sender's rank, tag, and the size of the received data. By dynamically allocating just the necessary space for each data transmission, this guarantees effective memory usage.

Assessed Workshop Task

5. The file "WarAndPeace.txt" on Canvas contains the entire text of the book "War and Peace" by Leo Tolstoy. Write an MPI program to count the number of times each letter of the alphabet occurs in the book. Count both the upper case and the lowercase as the same. Ignore any letter with accents such as " é " and so on.

Your MPI program should work with any number of processes from 1 to 100 processes. Only Process rank 0 (zero) should read in the file and send the appropriate chunk of file to each other process. The other processes should not read in the file.

You should submit this program as "workshoptask1.c" as part of your final portfolio submission. You can also upload it to the formative submission point for formative feedback.

```
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming
/Week 2$ nano
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming
/Week 2$ mpicc workshoptask1.c -o workshoptask1
ojashshakya@LenovoL0Q:/mnt/c/Users/Lenovo/OneDrive - University of Wolverhampton/2nd Year - 4th Sem/Distributed and Cloud Systems Programming
/Week 2$ mpiexec ./workshoptask1
Letter frequencies:
a: 201333
b: 34369
c: 60655
d: 117759
e: 311362
f: 54513
g: 50909
h: 166531
i: 170616
j: 2485
k: 20282
l: 96041
m: 61286
n: 183138
o: 188699
p: 44717
q: 2320
r: 146891
s: 162138
t: 224517
u: 63883
v: 26789
w: 58937
x: 4032
```

Explanation: This MPI-based C program divides the burden across several processes to determine how frequently each letter appears in War and Peace. Parts of the text are sent to other processes by the root process (rank 0), which reads the full file and determines the chunk size. The count_letters function, which increases counts for each alphabetic character (ignoring case), is used by every process, including the root, to count the letters in its allocated chunk. The root process then publishes the final letter frequencies after combining the local counts from each process into global counts using MPI_Reduce and a summing operation. This parallel method uses numerous processors to speed up the processing.

```

#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>

#define ALPHABET_SIZE 26
#define CHUNK_SIZE 4096

void count_letters(const char *text, int *local_counts) {
    for (int i = 0; text[i] != '\0'; i++) {
        if (isalpha(text[i])) {
            local_counts[tolower(text[i]) - 'a']++;
        }
    }
}

int main(int argc, char **argv) {
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int local_counts[ALPHABET_SIZE] = {0};
    int global_counts[ALPHABET_SIZE] = {0};
    char *buffer = NULL;
    long file_size = 0;

    if (rank == 0) {
        FILE *file = fopen("WarAndPeace.txt", "r");
        if (!file) {
            printf("Error: Could not open file.\n");
            MPI_Abort(MPI_COMM_WORLD, 1);
        }
        fseek(file, 0, SEEK_END);
        file_size = ftell(file);
        fseek(file, 0, SEEK_SET);
        buffer = (char *)malloc(file_size + 1);
        fread(buffer, 1, file_size, file);
        buffer[file_size] = '\0';
        fclose(file);
    }

    MPI_Bcast(&file_size, 1, MPI_LONG, 0, MPI_COMM_WORLD);
    int chunk_size = file_size / size;
    char *chunk = (char *)malloc((chunk_size + 1) * sizeof(char));

    MPI_Scatter(buffer, chunk_size, MPI_CHAR, chunk, chunk_size, MPI_CHAR, 0, MPI_COMM_WORLD);
    chunk[chunk_size] = '\0';

    count_letters(chunk, local_counts);

    MPI_Reduce(local_counts, global_counts, ALPHABET_SIZE, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        printf("Letter frequencies:\n");
        for (int i = 0; i < ALPHABET_SIZE; i++) {
            printf("%c: %d\n", 'a' + i, global_counts[i]);
        }
        free(buffer);
    }

    free(chunk);
    MPI_Finalize();
    return 0;
}

```